OS ORANGE 4 ASSIGNMENT

NAME: Mayukh Vinay

SRN: PES2UG24CS272

SEC: E

GITHUB LINK: https://github.com/mayukhvinay-sys/PES2UG24CS272-pes-vcs

PHASE 1:

1A.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

PHASE 2:

2A.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ find .pes/objects -type f
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ xxd .pes/objects/$(ls .pes/objects/ | head -1)/$(ls .pes/objects/$(ls .pes/objects/ | head -1)/ | head -1) | head -
20
xxd: Is a directory
```

PHASE 3:

3A.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ echo "hello" > file1.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ echo "world" > file2.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes add file1.txt file2.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  test_objects
  untracked:  test_tree
  untracked:  commit.h
  untracked:  tree.c
  untracked:  object.c
  untracked:  pes.h
  untracked:  Makefile
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  .gitignore
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  tree.h
  untracked:  pes.c
  untracked:  commit.c
```

3B.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776751207 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776751213 6 file2.txt
```

PHASE 4:

4A.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ echo "Hello" > hello.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes add hello.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes commit -m "Initial commit"
Committed: f555b2b88c99... Initial commit
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ echo "World" >> hello.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes add hello.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes commit -m "Add world"
Committed: 81d1eb1e94b2... Add world
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ echo "Goodbye" > bye.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes add bye.txt
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 97654b93c79b... Add farewell
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ ./pes log
commit 97654b93c79b6005070979d71cc71bcc17de96ff71cb84c221e798736f2fe3b
Author: Mayukh Vinay <mayukhvinay@gmail.com>
Date:   1776752670

    Add farewell


commit 81d1eb1e94b29dc1a6af931a43741c8be23fbdf6e0c7adfc71ef1e2a0789d337
Author: Mayukh Vinay <mayukhvinay@gmail.com>
Date:   1776752657

    Add world

Add farewell
1

commit f555b2b88c995de1dc550cfcee5e2ab0ccdb993a4bf01fb9d2ab6df386a9d378
Author: Mayukh Vinay <mayukhvinay@gmail.com>
Date:   1776752639

    Initial commit
iA
```

4B.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/81/d1eb1e94b29dc1a6af931a43741c8be23fbdf6e0c7adfc71ef1e2a0789d337
.pes/objects/97/654b93c79b60050070979d71cc71bcc17de96ff71cb84c221e798736f2fe3b
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f5/55b2b88c995de1dc550cfcee5e2ab0ccdb993a4bf01fb9d2ab6df386a9d378
.pes/refs/heads/main
```

4C.

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ cat .pes/refs/heads/main
97654b93c79b600500709979d71cc71bcc17de96ff71cb84c221e798736f2fe3b
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

INTEGRATION PART OUTPUTS:

```
mayukh@MVLaptop:~/PES2UG24CS272-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 522ffcd09a07... Initial commit

Log after first commit:
commit 522ffcd09a07cad042d131c0d1d46937bc31f0edd2134dc809e055bbd3b846e3
Author: Mayukh Vinay <mayukhvinay@gmail.com>
Date:   1776753168

    Initial commit


--- Second Commit ---
Committed: 66dd75426d58... Update file.txt

--- Third Commit ---
Committed: 9f0863821lbF... Add farewell

--- Full History ---
commit 9f0863821lbf41decaa36070a8762e1f06e0a824acdd66afdddb01e7bf9743fd
Author: Mayukh Vinay <mayukhvinay@gmail.com>
Date:   1776753168

    Add farewell

commit 66dd75426d587e268be24d843053026ef7ee4a1a3684afbf1ca8e007ce78afcc
Author: Mayukh Vinay <mayukhvinay@gmail.com>
Date:   1776753168

    Update file.txt

commit 522ffcd09a07cad042d131c0d1d46937bc31f0edd2134dc809e055bbd3b846e3
Author: Mayukh Vinay <mayukhvinay@gmail.com>
Date:   1776753168

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
9f0863821lbf41decaa36070a8762e1f06e0a824acdd66afdddb01e7bf9743fd

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69008bd9b0cc5934a610ab65da429b525361147fae7b5b922919e9a23143d
.pes/objects/53/2ffcd09a07cad042d131c0d1d46937bc31f0edd2134dc809e055bbd3b846e3
.pes/objects/58/a67ed1c161a4e89a11896881 0fe31a3992bef88d4c7c7e6b6e8957953f58d
.pes/objects/66/6dd75426d587e268be24d843053026ef7ee4a1a3684afbf1ca8e007ce78afcc
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b857b53b14b4b9b939df7d6ecc5321
.pes/objects/9f/b0863821lbf41decaa36070a8762e1f06e0a824acdd66afdddb01e7bf9743fd
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa13f74a981ce282730e85de0ccd92
.pes/objects/db/07a1451ca9594db f66d769b5863771d785efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

PHASE 5&6:

Q5.1 pes checkout needs to:

1. Resolve target commit: read .pes/refs/heads/.
2. Update HEAD to ref: refs/heads/ (or direct hash for detached mode).
3. Read target commit object, then its root tree.
4. Reconstruct working directory to match that tree:

- Write tracked files from blobs
- Create/remove directories
- Delete tracked files not present in target tree

5. Rebuild/update index to match checked-out tree metadata. Why complex:

- Must avoid clobbering uncommitted work.
- Needs recursive tree traversal and path-level conflict handling.
- Must be atomic enough that partial failures don't leave mixed states. Q5.2 Detect dirty-checkout conflict using index + object store:

1. For each tracked path that differs between current branch tree and target branch tree:

- Compare working file against index entry metadata (mtime, size) as fast filter.
- If metadata differs, hash current working file as blob candidate and compare with indexed blob hash.

2. If working file content != indexed blob and path also changes across branches, block checkout.
3. Also block when untracked working files would be overwritten by target tree paths. Core idea: "local modification" = working content diverges from index; "checkout danger" = same path diverges between current/target trees. Q5.3 Detached HEAD means HEAD points directly to a commit hash, not a branch ref. New commits are created normally, but no branch name advances, so those commits can become unreachable later. Recovery:
4. Find commit hash (from log, reflog-style history, or printed commit IDs).
5. Create/move a branch ref to it: e.g. write hash to .pes/refs/heads/recovered.
6. Set HEAD to ref: refs/heads/recovered to continue safely. Q6.1 GC algorithm:
7. Build root set from all refs (.pes/refs/heads/*, tags if any, detached HEAD hash if applicable).

8. Traverse graph (DFS/BFS):
- Commit -> parent commit(s)
- Commit -> root tree
- Tree -> child trees + blobs
3. Store visited hashes in a hash set (unordered_set-style) for O(1) membership.
4. Scan .pes/objects/**; delete any object hash not in reachable set. For 100,000 commits and 50 branches, visited commits are usually near ~100,000 total (branches overlap heavily), not 5,000,000. Total visited objects = commits + trees + blobs reachable from those commits; typically many more than commits, but still linear in reachable graph size. Q6.2 Why dangerous concurrently:
- Commit process may write new objects and later update branch ref.
- If GC runs between those steps, new objects may appear unreachable (no ref yet) and get deleted.
- Then commit finishes updating ref to hashes that GC already removed -> broken history/ reference. Example race:
1. Commit writes tree/blob/commit objects.
2. Before ref update, GC marks reachability from current refs only.
3. GC deletes those newly written objects as unreachable.
4. Commit updates refs/heads/main to deleted commit hash. How real Git avoids this:
- Uses object liveness protections (reflogs, grace periods, mtimes/prune windows).
- Uses locking/coordination and conservative pruning.
- Typically prunes only sufficiently old unreachable objects, reducing race risk drastically.